

Esercizio: Business Intelligence & Data Warehousing

Scenario: "TechStore Analytics"

L'azienda "TechStore", un e-commerce di elettronica in crescita, vuole migrare dal suo attuale sistema gestionale a un sistema di Business Intelligence per analizzare le vendite, i clienti e le spedizioni

. Tu sei il Data Engineer incaricato del progetto.

Sezione 1: Teoria e Architettura (Risposte Brevi)

Domanda 1:

OLTP vs OLAP Spiega la differenza fondamentale tra il database che TechStore usa per registrare gli ordini in tempo reale e il Data Warehouse che stai per costruire. Quale sistema usa uno schema normalizzato e quale denormalizzato?

Il database come techstore è concepito per non avere ambiguità e risonanza mentre il data warehouse si usa su mole di dati importante a cui serve eseguire query analitiche e avere uno storico

Domanda 2:

Architettura a Tre Livelli Descrivi brevemente i tre livelli (tier) dell'architettura di un Data Warehouse, spiegando cosa succede nel "Middle Tier".

Ci sono 3 livelli il bottom, middle e top tier il middle si occupa delle operazioni etl e di stoccaggio dati

Domanda 3:

Configurazione Metabase Per visualizzare i dati, TechStore ha scelto Metabase su Docker. Quali sono i passaggi fondamentali per configurare il file docker-compose.yaml affinché Metabase possa connettersi al database PostgreSQL?

Si prende il file docker-compose.yaml si specifica immagine e la porta di entrata in questo caso la (3100:3100) e si settano le variabili del sistema metabase

Sezione 2: Progettazione e Modellazione Dimensionale

Domanda 4:

Star Schema e Grain

Devi progettare lo schema per l'analisi delle vendite.

1. Definisci il **Grain** (livello di granularità) più appropriato per la tabella dei fatti, seguendo la regola di Kimball.

2. Disegna (o descrivi) lo **Star Schema**, elencando la *Fact Table* e almeno tre *Dimension Tables* necessarie.

Metterei al centro la tabella dei fatti, ad esempio, fact_vendite poi la tabella dim_prodotto(informazioni sul prodotto)

Dim_spedizione(informazioni sulla spedizione)

Dim_data(informazioni sulle date)

Domanda 5:

Gestione dello Storico (SCD) Il cliente "Mario Rossi" cambia il suo segmento da "Standard" a "Premium".

1. Se utilizziamo una **Slowly Changing Dimension (SCD) di Tipo 1**, cosa accade al record di Mario Rossi? Sovrascrive il dato precedente in questo caso da standard a premium la vecchia informazione viene sostituita completamente con quella nuova

2. Se utilizziamo una **SCD di Tipo 2**, come viene gestita la modifica e quali colonne aggiuntive sono necessarie nella tabella dim_cliente per supportarla?

Viene fleggata la modifica, rendendo il dato precedente valido fino alla data di fine e poi il nuovo valore verrà fleggato come valido da is_current in poi, inoltre, ci serve prendere il data_inizio,data_fine, is_current

Domanda 6: Chiavi Surrogate Perché nel Data Warehouse di TechStore dovremmo usare una **Surrogate Key** (es. cliente_sk) invece di usare direttamente la chiave primaria del gestionale (es. cliente_id)? Elenca almeno due vantaggi.

I vantaggi delle surrogate dalle primarie sono nel primo caso permette di isolare nel datawarehouse la chiave e il secondo vantaggio + importante è quello di avere lo storico di quel determinato dato

Sezione 3: Implementazione Pratica (SQL & ETL)

Domanda 7:

Query OLAP Scrivi una query SQL per calcolare i **ricavi totali** per ogni **categoria** di prodotto nell'anno **2024**. I risultati devono essere ordinati dal ricavo più alto al più basso. Assumi le seguenti tabelle: fact_vendite, dim_prodotto, dim_data.

```
Select dp.categoria
sum (fv.ricavi) as ricavi_totali
From fact_vendite as fv
JOIN dim_prodotto dp ON f.prodotto_key = dp.prodotto_key
JOIN dim_data dd ON f.data_key = dd.data_key
Where
Group by dp.categoria,
Orded by ricavi_totali asc,
Dd.anno=2024
```

Domanda 8:

Processo ETL con dblink Spiega come utilizzeresti l'estensione **dblink** di PostgreSQL per popolare la dimensione `dim_cliente` partendo dal database operativo (OLTP). Scrivi un esempio concettuale della query di estrazione.

Domanda 9:

Tabelle Pivot Il reparto marketing ti chiede un'analisi rapida per vedere quante vendite ci sono state per ogni combinazione di "Fascia d'Età" (righe) e "Categoria Prodotto" (colonne). Spiega come una **Tabella Pivot** trasforma i dati da un formato "lungo" a un formato "largo" per rispondere a questa domanda.

Essendo che la tabella pivot permette la visualizzazione dell'aggregazione tra la tabella `fact_vendite` e `dim_prodotto` allarga sulle righe permettendo una lettura + comprensibile e precisa

Soluzioni e Criteri di Valutazione (Per l'Istruttore)

Di seguito le risposte attese basate sulle fonti:

R1 (OLTP vs OLAP): L'OLTP è ottimizzato per transazioni veloci (INSERT, UPDATE) e usa uno schema normalizzato per evitare ridondanze

. L'OLAP è ottimizzato per l'analisi (SELECT aggregate) su dati storici e usa uno schema denormalizzato (Star Schema) per performance di lettura migliori,

R2 (Architettura):

1. *Bottom Tier*: Sorgenti dati (OLTP, API).

2. *Middle Tier (DW Core)*: Il database analitico dove avvengono ETL e stoccaggio dati ottimizzato.

3. *Top Tier*: Strumenti di presentazione/BI (es. dashboard)

,

.

R3 (Docker Metabase): Bisogna definire il servizio metabase nel `docker-compose.yaml`, specificare l'immagine (`metabase/metabase:latest`), mappare le porte (es. 3100:3000) e configurare le variabili d'ambiente per il database interno di Metabase (`MB_DB_TYPE`, `MB_DB_DBNAME`, ecc.)

.

R4 (Grain e Star Schema):

1. *Grain*: "Una riga per ogni prodotto venduto in un ordine" (o il "beep" alla cassa)

,

.

2. *Schema*: `fact_vendite` al centro. Dimensioni: `dim_data` (tempo), `dim_prodotto` (cosa), `dim_cliente` (chi), `dim_canale` (dove)

,

.

R5 (SCD):

1. *Tipo 1*: Sovrascrittura. Il vecchio dato "Standard" viene perso e sostituito da "Premium"

.

2. *Tipo 2*: Aggiunta nuova riga. Il vecchio record viene "chiuso" (`data_fine` impostata) e si inserisce una nuova riga con i nuovi dati e una nuova chiave surrogata. Colonne necessarie: `data_inizio`, `data_fine`, `is_current`

,

.

R6 (Chiavi Surrogate): Le chiavi surrogate isolano il DW dai cambiamenti nel sistema sorgente, permettono di gestire lo storico (più righe per lo stesso cliente_id in SCD Tipo 2) e sono più performanti (interi piccoli) rispetto a chiavi naturali complesse

,
.

R7 (SQL):

```
SELECT
    dp.categoria,
    SUM(f.ricavi) AS ricavi_totali
FROM fact_vendite f
JOIN dim_prodotto dp ON f.prodotto_key = dp.prodotto_key
JOIN dim_data dd ON f.data_key = dd.data_key
WHERE dd.anno = 2024
GROUP BY dp.categoria
ORDER BY ricavi_totali DESC;
```

(Fonte logica query:

)

R8 (ETL dblink): dblink permette di eseguire query su un database remoto come se fosse locale

. Esempio:

```
INSERT INTO dim_cliente (...)
SELECT * FROM dblink('dbname=techstore ...', 'SELECT id, nome FROM clienti')
AS t(id int, nome text);
```

(Fonte codice:

)

R9 (Pivot): La tabella pivot aggrega i dati raggruppando i valori unici di una colonna (Fascia Età) come intestazioni di riga e i valori unici di un'altra (Categoria) come intestazioni di colonna, calcolando un valore (es. conteggio o somma) all'incrocio, rendendo leggibili le relazioni tra variabili